

Farafina AI

L'IA souveraine au service des langues locales — Dioula & Mooré

Document de description du projet — Hackathon 2026 — Burkina Faso

1. Le problème

Au Burkina Faso, une grande partie de la population parle le Dioula ou le Mooré au quotidien, mais l'administration, ses documents et ses services en ligne fonctionnent exclusivement en français. Pour un citoyen peu lettré ou ne lisant pas l'alphabet latin, comprendre un acte de naissance, demander un casier judiciaire ou simplement savoir quelles démarches accomplir devient un obstacle majeur qui nécessite un intermédiaire humain, avec les coûts et les risques que cela implique.

2. La solution

Farafina AI est un assistant administratif multicanal (application web + bot Telegram) qui parle les langues locales. Il combine reconnaissance vocale, traduction automatique, synthèse vocale et un grand modèle de langage pour permettre à un usager de :

- **Parler** en Dioula, en Mooré ou en français, et recevoir une réponse vocale dans sa langue ;
- **Photographier un document administratif** (ou envoyer un PDF) et se le faire expliquer simplement, à voix haute, dans sa langue ;
- **Poser des questions** sur les démarches administratives et obtenir des réponses concrètes ;
- **Demander un casier judiciaire de bout en bout** : l'utilisateur envoie ses deux pièces (acte de naissance + CNIB/passeport), l'IA en extrait les informations, pose uniquement les questions restantes en conversation, remplit et soumet automatiquement le formulaire en ligne, puis livre le récépissé PDF directement dans la discussion.

Chaque message de l'interface est affiché en langue locale avec la traduction française entre parenthèses, et accompagné d'un message vocal pré-généré : les usagers qui ne lisent pas l'alphabet latin entendent le contenu des menus avant de choisir un bouton, sans temps d'attente.

3. Architecture technique

Le système est un monorepo déployé en plusieurs services indépendants, chacun dimensionné pour ses propres contraintes (mémoire, binaire navigateur, exécution persistante) :

Service	Rôle	Hébergement
backend (Next.js)	Application web, API, bot Telegram (webhook), orchestration LLM (Gemini 2.5 Flash), site DÉMO e-casier	Vercel
model-service	Traduction fr ↔ dyu/mos (NLLB-200-3.3B ou modèles GO AI Corporation)	HF Space Docker (~16 Go)
asr-service	Reconnaissance vocale Dioula/Mooré/français (Meta Omnilingual ASR — omniASR-LLM-7B)	HF Space Docker
tts-service	Synthèse vocale (facebook/mms-tts-dyu et mms-tts-mos), sortie OGG/Opus compatible Telegram	HF Space Docker
automation-service	Navigateur headless Playwright qui remplit le formulaire e-casier DÉMO et récupère le récépissé PDF	HF Space Docker

Cette séparation en Spaces est née d'une contrainte réelle : charger le modèle de traduction (~13 Go) et les modèles vocaux dans un même conteneur de 16 Go provoquait des plantages mémoire. Chaque push sur la branche principale redéploie automatiquement les services concernés via GitHub Actions.

4. Modèles d'IA utilisés

Fonction	Modèle	Remarques
Compréhension & génération	Google Gemini 2.5 Flash	Simplification de documents, réponses aux questions, extraction structurée des pièces d'identité
Traduction	facebook/nllb-200-3.3B ou goaicorp/{dyu,mos}-translation	Deux « stacks » commutables ; les modèles GO AI (CC-BY-NC 4.0) sont spécialisés par langue
Reconnaissance vocale	facebook/omniASR-LLM-7B	Couvre nativement le Dioula et le Mooré ; fallback local facebook/mms-1b-all
Synthèse vocale	facebook/mms-tts-dyu / mms-tts-mos	Encodage OGG/Opus pour lecture native comme message vocal Telegram

5. Le flux « casier judiciaire » de bout en bout

1. L'utilisateur choisit « Casier judiciaire » dans le menu du bot Telegram.
2. Il envoie une photo/PDF de son acte de naissance, puis de sa CNIB ou de son passeport ; Gemini en extrait les champs lisibles (nom, prénoms, date et lieu de naissance, filiation, numéro de pièce...) sans jamais deviner une valeur illisible.
3. Le bot ne pose **que** les questions restantes (domicile, profession, téléphone, région/province/commune de naissance...), une par une, en langue locale avec traduction française.
4. Un navigateur headless (automation-service) remplit et soumet automatiquement le formulaire du site e-casier DÉMO, y compris les listes déroulantes en cascade et le paiement simulé.
5. Le récépissé PDF généré est livré à l'utilisateur dans la conversation, avec relance automatique en cas d'échec d'envoi.

*Garde-fou éthique : l'automatisation ne cible **jamais** le vrai site gouvernemental. Un clone de démonstration complet (accueil, formulaire en 5 étapes, suivi de demande, génération de récépissé) a été développé et affiche en permanence un bandeau « DÉMO — données fictives ». Le code refuse explicitement toute URL pointant vers ecasier-judiciaire.gov.bf.*

6. Fiabilité et qualité

- Quotas d'utilisation quotidiens avec mise à jour atomique en base (sans condition de course), et crédits payants simulés ;
- Audio des menus pré-généré au déploiement : aucun temps d'attente TTS pour les messages prévisibles ;
- Temps de réponse affiché à l'utilisateur sur chaque réponse (« Répondu en X,X s ») ;
- Livraison fiable des documents (3 tentatives avec backoff), nettoyage automatique des fichiers audio côté serveur, sessions conversationnelles avec expiration ;
- Suites de tests (Vitest côté backend, pytest côté model-service), typage strict TypeScript, lint sans erreur.

7. Liens

Ressource	URL
Application web (traducteur)	https://hackton-locallang.vercel.app
Démo e-casier (services gouvernementaux)	https://hackton-locallang.vercel.app/demo
Bot Telegram	https://t.me/Africalangbot (@Africalangbot)
Dépôt GitHub	https://github.com/Achraf-cyber/hackton-locallang
Space traduction	https://achrafcyber-model-service.hf.space
Space reconnaissance vocale	https://achrafcyber-asr-service.hf.space
Space synthèse vocale	https://achrafcyber-tts-service.hf.space